Mithril in Game-based Syntax

Wuyunsiqin Bingsheng Zhang
Zhejiang University
oyun@zju.edu.cn bingsheng@zju.edu.cn

November 25, 2025

1 Primitives

1.1 BLS Signature Scheme

Let λ be the security parameter. We assume a bilinear group generation algorithm which on input 1^λ outputs

$$(q, G_1, G_2, G_T, g_1, g_2, e),$$

where G_1, G_2, G_T are cyclic groups of prime order q, $g_1 \in G_1$, $g_2 \in G_2$ are generators, and $e : G_1 \times G_2 \to G_T$ is a bilinear pairing. Let $H : \{0,1\}^* \to G_1$ be a hash function modeled as a random oracle.

We write the BLS signature scheme as

$$\Pi_{\mathsf{BLS}} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{Sign}, \mathsf{Ver}).$$

- $pp \leftarrow \mathsf{Setup}(1^\lambda)$: run the group generator to obtain $(q, G_1, G_2, G_T, g_1, g_2, e)$ and fix H. Output public parameters

$$\mathsf{pp} = (q, G_1, G_2, G_T, g_1, g_2, e, H).$$

- $(sk, vk) \leftarrow \mathsf{KGen}(\mathsf{pp})$: sample $sk \xleftarrow{\$} \mathbb{Z}_q$ and set

$$vk = g_2^{sk} \in G_2.$$

  Output the key pair (sk, vk).

- $\sigma \leftarrow \mathsf{Sign}(\mathsf{pp}, sk, msg)$: compute $h \leftarrow H(msg) \in G_1$ and set

$$\sigma = h^{sk} \in G_1.$$

  Output σ.

- $b \leftarrow \mathsf{Ver}(\mathsf{pp}, vk, msg, \sigma)$, where $b \in \{0,1\}$: compute $h \leftarrow H(msg)$ and output

$$b = 1 \text{ iff } e(\sigma, g_2) = e(h, vk).$$

1.2 Merkle Tree

Let $H_{\mathsf{MT}} : \{0,1\}^* \to \{0,1\}^\lambda$ be a collision-resistant hash function.

We denote the Merkle tree primitive by

$$\Pi_{\mathsf{MT}} = (\mathsf{Create}, \mathsf{Check}).$$

- $\mathsf{avk} \leftarrow \mathsf{Create}(\mathbf{vk}, \mathbf{w})$: on input a vector of verification keys $\mathbf{vk} = (vk_1, \ldots, vk_N)$ and a matching stake vector $\mathbf{w} = (w_1, \ldots, w_N)$, set leaves $\ell_i := H_{\mathsf{MT}}(vk_i, w_i)$ and output the Merkle root avk.
- $b \leftarrow \mathsf{Check}(\mathsf{avk}, vk, w, path)$: on input a root avk, a pair (vk, w), and an authentication path $path$, recompute the root from $\ell := H_{\mathsf{MT}}(vk, w)$ and $path$. Output (b = 1) iff the recomputed root equals avk, and (b = 0) otherwise.

2 Stake-based threshold multi-signature

2.1 Syntax

We model Mithril as an α-STMS whose acceptance is determined on a *stake* basis via a "t wins in n lotteries" parametrization.

$$\Pi_{\mathsf{STMS}} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{AKey}, \mathsf{ACheck}, \mathsf{Sig}, \mathsf{ASig}, \mathsf{AVer}).$$

$\mathsf{Setup}(1^\lambda) \to (\mathsf{pp}, n)$**.** On input a security parameter, outputs system parameters pp and a lottery parameter $n \in \mathbb{N}$ (the number of lotteries).

$\mathsf{KGen}(\mathsf{pp}) \to (sk, vk)$**.** Generates a signing/verification key pair.

$\mathsf{AKey}(\mathbf{vk}, \mathbf{w}) \to \mathsf{avk}$**.** Given a list of verification keys $\mathbf{vk} := (vk_1, \ldots, vk_N)$ and a matching stake vector $\mathbf{w} := (w_1, \ldots, w_N)$, deterministically produces an aggregate verification key avk. //todo: define N

$\mathsf{ACheck}(\mathsf{avk}, \mathbf{vk}, \mathbf{w}) \to b \in \{0,1\}$**.** Deterministically checks whether avk is the correct aggregate key for $(\mathbf{vk}, \mathbf{w})$.

$\mathsf{Sig}(vk, w, sk, j, m) \to \sigma$ **.** Attempts to produce a local proof σ for message n and lottery index $j \in \{1, \ldots, n\}$ on behalf of a user with key pair (sk, vk) and stake w. It outputs $\perp$ if the user is ineligible for (m, j) under stake w.

$\mathsf{ASig}(\mathbf{vk}, \mathbf{w}, m, \{(vk_i, \sigma_i)\}_{i=1}^t) \to \tau$**.** Given $(\mathbf{vk}, \mathbf{w})$, a message m, and a multiset of signer/signature pairs with pairwise-distinct lottery indices, aggregates them into a threshold multi-signature τ; outputs $\perp$ if inputs are insufficient or inconsistent.

$\mathsf{AVer}(\mathsf{avk}, m, \tau, W) \to b \in \{0,1\}$**.** Given a message m, an aggregate key avk, an aggregate signature τ, and the total stake $W = \sum_i w_i$, deterministically accepts ($b = 1$) iff τ is a valid threshold multi-signature on m under avk and W.

2.1.1 Correctness

Aggregate-key correctness. For any $(\mathbf{vk}, \mathbf{w}) \in \{\}$,

$$\mathsf{avk} \leftarrow \mathsf{AKey}(\mathbf{vk}, \mathbf{w}) \quad \Longrightarrow \quad \Pr\left[\mathsf{ACheck}(\mathsf{avk}, \mathbf{vk}, \mathbf{w}) = 1\right] = 1 - \mathsf{negl}(\lambda).$$

//todo: define negl

Local-signature correctness. For any honest key pair (sk, vk) with stake w, any $j \in \{1, \ldots, \mu\}$ and message m, if $\pi = (j, \sigma, vk, w) \leftarrow \mathsf{Sig}(vk, w, sk, j, m) \neq \perp$, then

$$e(\sigma, g_2) = e(H(m), vk)$$

except with negligible probability. (Equivalently, σ passes BLS verification.)

Aggregate-signature correctness (completeness and acceptance). Consider an execution where the set of honest users and their stakes are fixed, and let $E(m)$ denote the set of all (index, user) pairs that are *eligible* on topic $m.\mathsf{topic}$ (eligibility depends on $(\sigma, j, m.\mathsf{topic})$ through $Eval$). Let the adversary choose a multiset of local proofs $\{\pi_i\}$ that includes, for at least one collection of indices $J \subseteq \{1, \ldots, t\}$, *all* local proofs of honest users for indices in J (completeness for one consistent index set), with pairwise-distinct indices. If $\tau \leftarrow \mathsf{ASig}(\mathbf{vk}, \mathbf{w}, m, \{\pi_i\})$ is produced, then

$$\mathsf{AVer}(\mathsf{avk}, m, \tau, W) = 1$$

except with negligible probability. Intuitively, whenever the aggregator includes a complete set of honest eligible local proofs for some consistent set of lottery indices, the resulting aggregate is accepted.

2.2 Security

Stake-based unforgeability α-UF-CMA. In the unforgeability experiment, the challenger runs Setup, generates honest keys (sk_i, vk_i), fixes $\mathbf{w}$, computes avk, and gives $(\mathsf{pp}, \mathbf{vk}, \mathbf{w}, \mathsf{avk})$ to the adversary $\mathcal{A}$. The adversary can:

- query *signing oracles* for any honest user i, any j, and any message m, receiving $\mathsf{Sig}(vk_i, w_i, sk_i, j, m)$;
- *corrupt* users to obtain sk_i (and then act under those keys).

Let C^* be the set of corrupted users. Let H^* be the set of honest users that *participated* on the forged topic (made at least one signing query for the challenge

topic) under the forged aggregate key. The experiment *charges* only successful eligibility (i.e., $\mathsf{Sig} \neq \perp$) and enforces the stake bound

$$\sum_{i \in C^* \cup H^*} w_i \;\leq\; \alpha\, W.$$

Finally, $\mathcal{A}$ outputs $(\mathbf{vk}^*, \mathsf{avk}^*, m^*, \tau^*)$; it wins if

$$\mathsf{ACheck}(\mathsf{avk}^*, \mathbf{vk}^*, \mathbf{w}) = 1, \quad \mathsf{AVer}(\mathsf{avk}^*, m^*, \tau^*, W) = 1,$$

and τ^* is *fresh*, i.e., it cannot be obtained by aggregating local proofs returned (or derivable) from oracle responses. An STMS is α-$\mathsf{UF\text{-}CMA}$ secure if every PPT adversary wins with only negligible probability.

Strengthened unforgeability α-UF-CMA-S. As above, but the adversary may first register arbitrary public keys (and corresponding stake placement) before AKey is run (key-substitution/registration stage). The rest of the game is identical. Security requires negligible winning probability for all PPT adversaries.

Notes on the model.

- Eligibility depends on the topic but not the body. This prevents grinding by changing body while preserving eligibility.
- We adopt the "do not charge unsuccessful attempts" rule: an ineligible honest user (output $\perp$) is *not* counted toward the adversarial stake budget; this mirrors the bridge paper's UF-CMA variant that balances probing versus corruption.
- Completeness in aggregate correctness is enforced against one consistent set of indices; pairwise-distinct indices in ASig prevent double-counting a lottery win.

2.3 Instantiation

We write the Mithril scheme as

$$\Pi_{\mathsf{Mithril}} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{AKey}, \mathsf{ACheck}, \mathsf{Sig}, \mathsf{ASig}, \mathsf{AVer}).$$

- $(\mathsf{pp}) \leftarrow \mathsf{Setup}(1^\lambda)$:
  run $\Pi_{\mathsf{BLS}}.\mathsf{Setup}(1^\lambda)$ to obtain $(q, G_1, G_2, G_T, g_1, g_2, e, H)$
  fix a dense mapping / eligibility function

  $$Eval : (msg, index, \sigma) \mapsto \{0,1\}^\lambda,$$

  a weighting function

  $$\varphi : \mathbb{N} \to [0,1],$$

e.g. $\varphi(w) = 1 - (1-f)^w$.

Index range

$$\mu \in \mathbb{N},$$

a quorum or lottery parameter

$$k \in \{1, \ldots, \mu\}.$$

Output

$$\mathsf{pp} = (q, G_1, G_2, G_T, g_1, g_2, e, H, Eval, \varphi, \mu, k).$$

- $(sk, vk) \leftarrow \mathsf{KGen}(\mathsf{pp})$:
  run $\Pi_{\mathsf{BLS}}.\mathsf{KGen}(\mathsf{pp})$

  $$(sk, vk) \leftarrow \Pi_{\mathsf{BLS}}.\mathsf{KGen}(\mathsf{pp})$$

  Output (sk, vk).

- $\mathsf{avk} \leftarrow \mathsf{AKey}(\mathbf{vk}, \mathbf{w})$:
  given

  $$\mathbf{vk} = (vk_1, \ldots, vk_N),$$
  $$\mathbf{w} = (w_1, \ldots, w_N),$$

  set

  $$\mathsf{avk} \leftarrow \Pi_{\mathsf{MT}}.\mathsf{Create}(\mathbf{vk}, \mathbf{w}).$$

- $b \leftarrow \mathsf{ACheck}(\mathsf{avk}, \mathbf{vk}, \mathbf{w})$, where $b \in \{0, 1\}$:
  compute

  $$avk' \leftarrow \Pi_{\mathsf{MT}}.\mathsf{Create}(\mathbf{vk}, \mathbf{w}).$$

  output b=1 iff avk=avk'.

- $\pi \leftarrow \mathsf{Sig}(\mathsf{pp}, vk, w, sk, j, m)$:
  compute the BLS signature

  $$\sigma \leftarrow \Pi_{\mathsf{BLS}}.\mathsf{Sign}(\mathsf{pp}, sk, m),$$

  compute the eligibility value

  $$ev \leftarrow Eval(\sigma, j, m),$$

  and check the threshold

  $$ev \ < \ \lfloor \varphi(w) \cdot 2^{\lambda} \rfloor.$$

  If true, output

  $$\pi = (j, \sigma, vk, w).$$

  Otherwise output $\perp$.

- $\tau \leftarrow \mathsf{ASig}(\mathsf{pp}, m, \{\pi_i\}_{i=1}^{t})$:
  parse each
  $$\pi_i = (j_i, \sigma_i, vk_i, w_i),$$
  with pairwise distinct
  $$j_i \in \{1, \ldots, \mu\}.$$
  Set
  $$\sigma^\star \;=\; \prod_{i=1}^{t} \sigma_i \;\in\; G_1,$$
  $$vk^\star \;=\; \prod_{i=1}^{t} vk_i \;\in\; G_2.$$
  Derive a batching seed
  $$e_\sigma \;\leftarrow\; H\big(\sigma_1 \,\|\, \cdots \,\|\, \sigma_t\,\big),$$
  and coefficients for $i = 1, \ldots, t$
  $$\rho_i \;\leftarrow\; \mathsf{H}_\lambda(e_\sigma \,\|\, i) \in \mathbb{Z}_q^*.$$
  Compute the batch aggregates
  $$\mu \;=\; \prod_{i=1}^{t} \sigma_i^{\rho_i} \;\in\; G_1,$$
  $$ivk \;=\; \prod_{i=1}^{t} vk_i^{\rho_i} \;\in\; G_2.$$
  Output
  $$\tau \;=\; (\,\sigma^\star,\, vk^\star,\, \mu,\, ivk,\, e_\sigma).$$
- $b \leftarrow \mathsf{AVer}(\mathsf{pp}, \mathsf{avk}, m, \tau)$, where $b \in \{0, 1\}$:
  parse
  $$\tau = (\sigma^\star, vk^\star, \mu, ivk, e_\sigma).$$
  Check that the indices are pairwise distinct. Compute
  $$h \leftarrow H(m).$$
  Verify the equation
  $$e(\sigma^\star, g_2) \;=\; e(h, vk^\star).$$
  Recompute the batching coefficients for $i = 1, \ldots, t$
  $$\rho_i \;\leftarrow\; \mathsf{H}(e_\sigma \,\|\, i) \in \mathbb{Z}_q^*,$$

and the corresponding aggregated public key

$$ivk' \;=\; \prod_{i=1}^{t} vk_i^{\rho_i} \;\in\; G_2.$$

Check consistency of the submitted aggregated key

$$ivk' \;=\; ivk.$$

Verify the batched equation

$$e(\mu, g_2) \;=\; e(h, ivk).$$

Output

$$b \;=\; 1 \text{ iff all checks above hold.}$$